

C++编程练习示例

以下是各编程练习的参考实现代码：

1. 字符串打印函数（含调用次数统计）

```
#include <iostream>
#include <cstring>
using namespace std;
// 静态变量记录调用次数，第二个参数非0时按次数打印
void printStr(const char* str, int flag = 0) {
    static int callCount = 0;
    callCount++;
    if (flag == 0) {
        cout << str << endl;
    } else {
        for (int i = 0; i < callCount; i++) {
            cout << str << endl;
        }
    }
}

int main() {
    printStr("First call");    // 打印1次
    printStr("Second call", 1); // 第2次调用，打印2次
    printStr("Third call", 1); // 第3次调用，打印3次
    return 0;
}
```

2. CandyBar 结构处理

```
#include <iostream>
#include <cstring>
using namespace std;
struct CandyBar {
    char brand[40];
    double weight;
    int calories;
};
```

```

};
// 设置CandyBar成员（带默认参数）
void setCandy(CandyBar& cb, const char* brand = "Millennium Munch",
double weight = 2.85, int calories = 350) {
    strcpy(cb.brand, brand);
    cb.weight = weight;
    cb.calories = calories;
}
// 显示CandyBar内容
void showCandy(const CandyBar& cb) {
    cout << "Brand: " << cb.brand << endl;
    cout << "Weight: " << cb.weight << endl;
    cout << "Calories: " << cb.calories << endl << endl;
}
int main() {
    CandyBar cb1, cb2;
    setCandy(cb1);           // 使用默认值
    setCandy(cb2, "Snickers", 4.5, 250); // 自定义值
    showCandy(cb1);
    showCandy(cb2);
    return 0;
}

```

3. 字符串转大写

```

#include <iostream>
#include <string>
#include <cctype>
using namespace std;
// 将字符串转为大写
void toUpper(string& s) {
    for (char& c : s) {
        c = toupper(c);
    }
}
int main() {
    string s;
    cout << "Enter a string (q to quit): ";
    while (getline(cin, s) && s != "q") {

```

```

toUpper(s);
cout << s << endl;
cout << "Next string (q to quit): ";
}
cout << "Bye." << endl;
return 0;
}

```

4. stringy 结构处理

```

#include <iostream>
#include <cstring>
using namespace std;
struct stringy {
    char* str; // 指向字符串
    int ct;    // 字符串长度 (不含'\0')
};
// 为stringy分配空间并复制字符串
void set(stringy& sy, const char* str) {
    sy.ct = strlen(str);
    sy.str = new char[sy.ct + 1];
    strcpy(sy.str, str);
}
// 打印stringy (默认打印1次)
void show(const stringy& sy, int n = 1) {
    for (int i = 0; i < n; i++) {
        cout << sy.str << endl;
    }
}
// 打印字符串 (默认打印1次)
void show(const char* str, int n = 1) {
    for (int i = 0; i < n; i++) {
        cout << str << endl;
    }
}
int main() {
    stringy beany;
    char testing[] = "Reality isn't what it used to be.";
    set(beany, testing);
}

```

```
show(beany);
show(beany, 2);
testing[0] = 'D';
testing[1] = 'u';
show(testing);
show(testing, 3);
show("Done!");
delete[] beany.str; // 释放动态内存
return 0;
}
```

5. 模板函数 max5（5 元素数组最大值）

```
#include <iostream>
using namespace std;
// 模板函数：返回5元素数组的最大值
template <typename T>
T max5(const T arr[5]) {
    T maxVal = arr[0];
    for (int i = 1; i < 5; i++) {
        if (arr[i] > maxVal) maxVal = arr[i];
    }
    return maxVal;
}
int main() {
    int intArr[5] = {1, 3, 5, 2, 4};
    double doubleArr[5] = {3.14, 1.59, 2.65, 5.89, 0.77};
    cout << "int数组最大值: " << max5(intArr) << endl;    // 输出5
    cout << "double数组最大值: " << max5(doubleArr) << endl; // 输出5.89
    return 0;
}
```

6. 模板函数 maxn（任意长度数组最大值）

```
#include <iostream>
#include <cstring>
using namespace std;
```

```

// 模板函数：返回任意长度数组的最大值
template <typename T>
T maxn(const T arr[], int n) {
    T maxVal = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > maxVal) maxVal = arr[i];
    }
    return maxVal;
}

// 具体化：返回char指针数组中最长字符串的地址
template <>
const char* maxn(const char* arr[], int n) {
    int maxLen = strlen(arr[0]);
    int idx = 0;
    for (int i = 1; i < n; i++) {
        if (strlen(arr[i]) > maxLen) {
            maxLen = strlen(arr[i]);
            idx = i;
        }
    }
    return arr[idx];
}

int main() {
    int intArr[6] = {3, 7, 2, 8, 1, 5};
    double doubleArr[4] = {2.2, 5.5, 1.1, 3.3};
    const char* strArr[5] = {"apple", "banana", "cherry", "date", "elderberry"};
    cout << "int数组最大值: " << maxn(intArr, 6) << endl;    // 输出8
    cout << "double数组最大值: " << maxn(doubleArr, 4) << endl; // 输出5.5
    cout << "最长字符串: " << maxn(strArr, 5) << endl;      // 输出"elderberry"
    return 0;
}

```

7. 模板函数 SumArray（数组总和）

```

#include <iostream>
using namespace std;
struct debts {
    char name[50];
    double amount;

```

```

};
// 模板函数：返回数组元素总和
template <typename T>
T SumArray(const T arr[], int n) {
    T sum = 0;
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    return sum;
}
// 模板函数：处理指针数组（如debt的amount指针）
template <typename T>
T SumArray(T* const arr[], int n) {
    T sum = 0;
    for (int i = 0; i < n; i++) {
        sum += *arr[i];
    }
    return sum;
}
int main() {
    int things[6] = {13, 31, 103, 301, 310, 130};
    debts mrE[3] = {
        {"Ima Wolfe", 2400.0},
        {"Ura Foxe", 1300.0},
        {"Iby Stout", 1800.0}
    };
    double* pd[3];
    for (int i = 0; i < 3; i++) {
        pd[i] = &mrE[i].amount; // 指向amount成员
    }
    cout << "things数组总和: " << SumArray(things, 6) << endl; // 输出888
    cout << "债务总和: " << SumArray(pd, 3) << endl; // 输出5500.0
    return 0;
}

```

以上代码均按题目要求实现，包含模板、引用、const 修饰、动态内存管理等特性，可直接编译运行。

（注：文档部分内容可能由 AI 生成）